

Multiple Choice — Decisiones Arquitectónicas

Preguntas de opción múltiple con respuesta justificada y distractores explicados

Ingeniería de Software II · ITBA · 2026-1C · Práctica de cátedra

RESUMEN

Banco de **multiple choice** sobre decisiones arquitectónicas concretas: qué BD, qué mecanismo de seguridad, qué patrón de comunicación, cómo resolver concurrencia, CAP e idempotencia. Cada pregunta lleva contexto, 4 opciones, la **respuesta canónica** y por qué cada distractor falla. Las marcadas (zona de trade-off) admiten más de una respuesta defendible si se justifica bien (source: mc-decisiones-arquitectonicas.md).

REGLA DE ORO

Cómo autoevaluarse: tapá el callout de respuesta, leé contexto + opciones, elegí y escribí el "porqué" en 1–2 líneas, y recién ahí comparás. Si la opción coincide pero el "porqué" no, ese es el déficit a cerrar (source: mc-decisiones-arquitectonicas.md).

§1 Persistencia, seguridad, replicación, CAP, idempotencia

Pregunta 1 — Chat corporativo interno

Contexto. Chat tipo Slack interno: 100.000 empleados, ~50 M mensajes/día, query dominante = "últimos 50 mensajes de la conversación X en orden inverso por timestamp". Append-only, retención indefinida, ráfagas de decenas de mensajes/seg en una sola conversación.

- a) PostgreSQL con índice compuesto sobre (`conversation_id`, `created_at` DESC).
- b) MongoDB con un documento por conversación que embebe todos los mensajes.
- c) **Cassandra particionada por `conversation_id`, clustering key `created_at` DESC.**
- d) Redis como almacén principal con listas por `conversation_id`.

CUÁNDO SÍ · VENTAJAS

Correcta: c. Patrón textual de Twitter Tweets/Timelines: query bounded por partición + ordenamiento temporal inverso + escrituras append-only de alta velocidad → wide-column con sharding por `conversation_id` y clustering temporal. Cassandra está diseñada para esto, con latencia sub-10 ms leyendo el head de la partición (Lakshman & Malik, 2009).

CUÁNDO NO • CUIDADO

Distractores: **a)** PostgreSQL particionado funciona pero se vuelve operacionalmente complejo a estos volúmenes (sharding manual, vacuum, índices que no entran en RAM) — los problemas que Twitter narra antes de migrar a Cassandra. **b)** documento monolítico choca con el máximo de 16 MB y reescribe el documento entero ante cada mensaje. **d)** Redis no es fuente de verdad y no escala el footprint indefinidamente.

TRADE-OFF

Zona de trade-off: para volúmenes ≤ 5 M msgs/día, PostgreSQL particionado (a) es perfectamente válido. La frontera práctica está cerca del volumen del enunciado (source: mc-decisiones-arquitectonicas.md).

Pregunta 2 — Telemetría de flota

Contexto. 5.000 camiones enviando GPS + telemetría cada 10 s. ~43 M eventos/día. Queries exclusivamente temporales (trayectoria por rango, agregados por ventana, alertas históricas). Histórico 2 años con downsampling.

- a) PostgreSQL particionado por mes.
- b) MongoDB con un documento por evento.
- c) **Time-series DB especializada (TimescaleDB, InfluxDB, Prometheus).**
- d) Cassandra particionada por `camion_id`.

CUÁNDO SÍ • VENTAJAS

Correcta: **c.** El driver es el *patrón de acceso temporal puro*: agregados por ventana, downsampling automático, retención por edad, compresión de columnas numéricas. TimescaleDB embebe esto en PostgreSQL con *hypertables* particionadas por tiempo; InfluxDB tiene *continuous queries* y *retention policies*.

CUÁNDO NO • CUIDADO

Distractores: **a)** PostgreSQL plano se rompe en `GROUP BY` masivos y no comprime tan eficiente. **b)** MongoDB no aprovecha la localidad temporal. **d)** Cassandra podría servir pero pierde las queries SQL analíticas y los operadores de tiempo nativos.

Pregunta 3 — Integración con domótica heterogénea

Contexto. Plataforma de domótica que controla luces, persianas, termostatos, cerraduras y cámaras de **distintas marcas con protocolos propios** (Zigbee, Z-Wave, Wi-Fi propietario, MQTT, REST). El núcleo no debe conocer los protocolos. Debe poder agregarse una marca nueva sin tocar el núcleo.

- a) Hierarchical Layers — cada protocolo es una capa.
- b) **Adapter por protocolo + bus interno con eventos canónicos.**
- c) Forwarder-Receiver entre el núcleo y cada dispositivo.
- d) Batch Sequential — recolectar comandos del día y enviarlos a la noche.

CUÁNDO SÍ • VENTAJAS

Correcta: b. Patrón textual del Ejercicio 2013 (control de fábrica con sensores serie): un *adapter por protocolo* encapsula la idiosincrasia de cada uno y traduce a un evento canónico interno que viaja por un bus pub-sub. Agregar marca = agregar adapter, núcleo intacto. Mejora **PORTABILITY** ([[Atributos de calidad]]).

CUÁNDO NO • CUIDADO

Distractores: a) Hierarchical Layers no resuelve heterogeneidad de protocolos. **c)** Forwarder-Receiver sirve entre dos pares conocidos — acá hay N protocolos heterogéneos y N+1 dispositivos. **d)** Batch Sequential no aplica al perfil real-time.

Pregunta 4 — App de pagos QR con saldo offline

Contexto. Billetera virtual: la app debe descontar saldo y aprobar pagos **sin conexión** en pagos chicos (~\$5.000), con reconciliación al volver la conectividad. Pagos > \$50.000 sí requieren online obligatorio.

- a) CP estricto — sin conexión, el pago se rechaza siempre.
- b) **AP con caché local del saldo en el celular y reconciliación posterior contra el servidor; idempotency key por transacción.**
- c) CA — confiar en que la red nunca particiona.
- d) Eventual consistency con last-write-wins ignorando el saldo del servidor.

CUÁNDO SÍ • VENTAJAS

Correcta: b. Decisión textual del mock TelePeaje y de [[Teorema CAP]]: cuando hay un requerimiento físico (zona sin red) que no admite esperar consenso global, se elige AP local con caché y reconciliación posterior. El umbral de \$50.000 modela el principio: a montos bajos el costo del fraude por divergencia es bajo → AP; a montos altos se exige CP (online obligatorio). Es el patrón offline-first de [[Caso Compraventa de Acciones|las cabinas]] y los POS de retail.

CUÁNDO NO • CUIDADO

Distractores: a) CP estricto rompe el caso de uso. **c)** CA niega la realidad de Brewer: la red particiona. **d)** LWW puro sin reconciliación de saldo genera doble gasto.

Pregunta 5 — Imágenes de catálogo de e-commerce

Contexto. 50 M publicaciones, 3–8 imágenes/publicación, ~400 KB c/u, hoy en columnas BYTEA de PostgreSQL. La base creció a 80 TB, backups inmanejables, queries lentas por bloqueo de páginas grandes en memoria.

- a) Mantenerlas en PostgreSQL con `pg_largeobject`.
- b) MongoDB con GridFS.
- c) **Object storage (S3, MinIO) + URL en PostgreSQL.**
- d) Redis con keys por `producto_id`.

CUÁNDO SÍ • VENTAJAS

Correcta: c. La naturaleza del dato manda (lección Compraventa): blobs grandes inmutables $\neq$ datos relacionales. Patrón canónico (Kleppmann, *DDIA*; Fowler, PoEAA): *separar el blob del metadato* — blobs en object storage (escalado horizontal nativo, bajo costo/TB, replicación geográfica, lifecycle policies para frío) y en la relacional sólo la URL + metadatos.

CUÁNDO NO • CUIDADO

Distractores: a) mantenerlo en PostgreSQL preserva el problema. b) GridFS es funcional pero pierde el beneficio de CDN y storage tierado. d) Redis es memoria, costoso para 80 TB de blobs raramente accedidos.

Pregunta 6 — Reserva concurrente del mismo turno (zona de trade-off)

Contexto. Dos pacientes hacen clic al mismo tiempo en el mismo slot médico. Contención baja (la mayoría de slots no se pelean), picos cuando hay especialista escaso. Monolito sobre PostgreSQL.

- a) **Optimistic locking con column `version` validada en el UPDATE.**
- b) No hacer nada en BD; resolver en código con `if slot.disponible` antes del UPDATE.
- c) Last-write-wins.
- d) Eventual consistency con resolución offline por mail.

CUÁNDO SÍ • VENTAJAS

Correcta: a (para baja contención). El lock optimista (Fowler, PoEAA, *Optimistic Offline Lock*) es canónico con baja contención: no bloquea filas en lecturas, el conflicto se detecta sólo al commit y se resuelve con error claro al perdedor.

CUÁNDO NO • CUIDADO

Distractores: b) race condition clásico: el `if` y el `UPDATE` no son atómicos. c) LWW permite que dos turnos se confirmen al mismo slot. d) eventual + email viola la UX y la integridad del calendario.

TRADE-OFF

Zona de trade-off: pessimistic locking (`SELECT FOR UPDATE`) es correcto y necesario si la contención sube, pero penaliza throughput. El enunciado dice "baja contención" $\rightarrow$ optimistic (source: mc-decisiones-arquitectonicas.md).

Pregunta 7 — Sincronización de app móvil offline

Contexto. App offline-first que sincroniza al volver online. La red móvil flaky puede hacer que un POST llegue al servidor pero la respuesta no vuelva al cliente, y el cliente reintente. Riesgo: nota duplicada.

- a) Detectar duplicados por hash del contenido.

- b) **Idempotency key (UUID) por operación; el servidor rechaza una segunda llamada con la misma key.**
- c) Bloquear reintentos en el cliente con timeout más largo.
- d) Last-write-wins.

CUÁNDO SÍ • VENTAJAS

Correcta: b. Patrón textual de Twitter Social Graphs (slide *Challenges*): "Write operations are idempotent: retry until success." El cliente genera un `idempotency_key` por operación lógica (no por intento); el servidor cachea el resultado de la primera ejecución y lo devuelve ante reintentos. Mismo patrón que Stripe (`Idempotency-Key`), Square, AWS API.

CUÁNDO NO • CUIDADO

Distractores: a) el hash falla si dos notas legítimas tienen el mismo texto. **c)** timeouts más largos no eliminan la causa raíz. **d)** LWW no resuelve la duplicación — sólo determina cuál gana si hay update sobre el mismo recurso.

Pregunta 8 — Defensa contra SQL injection

Contexto. Formulario público con búsqueda que concatena texto del usuario en SQL. El pen-test reveló vulnerabilidad. Backend Java sobre PostgreSQL.

- a) Encriptación at-rest.
- b) **Sanitización + prepared statements / consultas parametrizadas.**
- c) MFA en el login.
- d) HTTPS end-to-end.

CUÁNDO SÍ • VENTAJAS

Correcta: b. La amenaza es *inyección*; el mecanismo del catálogo ([[Mecanismos de seguridad]]) son **prepared statements**, que separan el código SQL de los datos a nivel driver y eliminan la posibilidad estructural de inyección, más sanitización defensiva. Lección de OWASP Top 10 (A03:2021 — Injection) y Clase 7.

CUÁNDO NO • CUIDADO

Distractores: a) at-rest protege storage, no auth ni queries. **c)** MFA protege login, no inyección. **d)** HTTPS protege el transporte, no el contenido del input.

Pregunta 9 — Log inmutable de auditoría compliance

Contexto. Log de auditoría financiera: inmutable, retención 10 años, reconstruible bit-a-bit ante el regulador, ~5 M entradas/día. Queries raras y dirigidas.

- a) PostgreSQL con UPDATE/DELETE revocados por permisos.
- b) **Event log append-only firmado por HSM, replicado a almacenamiento WORM (write-once read-many).**
- c) Redis con TTL de 10 años.

- d) MongoDB con `findAndModify` desactivado.

CUÁNDO SÍ • VENTAJAS

Correcta: b. Patrón textual del mock TelePeaje y de Compraventa: *event sourcing* sobre un log append-only **firmado** (no-repudio por HSM) y replicado a **WORM** — la inmutabilidad es propiedad del medio, no de un permiso revocable. Institucionaliza el "no-repudio físico" que exigen los reguladores financieros (Basilea, CNV).

CUÁNDO NO • CUIDADO

Distractores: a) y d) no son realmente inmutables: un DBA con privilegios o un exploit puede modificar; inmutabilidad por política, no por física. **c)** Redis es memoria volátil — viola la durabilidad de 10 años.

Pregunta 10 — Detección de sniping en subastas

Contexto. Detectar bots de sniping: "si un usuario realiza > 5 pujas en los últimos 10 s de una subasta de > 1 h, todas del mismo orden de magnitud, marcar". 2.000 subastas concurrentes; detección **en vivo**.

- a) Batch nocturno.
- b) SEP — cada puja evaluada individualmente contra reglas estáticas.
- c) **CEP — Complex Event Processing (Flink, Esper, Kafka Streams).**
- d) Map-Reduce cada 15 min.

CUÁNDO SÍ • VENTAJAS

Correcta: c. La regla tiene la firma textual de *CEP* ([[Estilos arquitectónicos|CEP]]): correlación de múltiples eventos en una ventana temporal con condiciones compuestas ("5 pujas / 10 s / mismo usuario / mismo orden de magnitud"). Motores reales: Apache Flink CEP, Esper, Kafka Streams con `windowed()`.

CUÁNDO NO • CUIDADO

Distractores: b) SEP evalúa eventos individuales — no puede correlacionar entre pujas distintas. **a)** batch nocturno y **d)** Map-Reduce 15-min violan el requisito de detección en vivo durante la subasta.

EN EL PARCIAL

En MC de escenario, la consigna oculta el driver dominante (volumen, patrón de acceso, requisito físico, amenaza). Identificarlo primero descarta 2–3 opciones de una. Para las **zona de trade-off**, la respuesta cambia con el ranking de atributos: justificá bajo qué supuestos cada alternativa sería válida (source: mc-decisiones-arquitectonicas.md).

FUENTE

Buschmann et al., POSA Vol. 1; Fowler, PoEAA (Optimistic Offline Lock, Identity Map); Kleppmann, Designing Data-Intensive Applications (caps. 3, 5, 6, 8); Bass, Clements, Kazman, SAIP; OWASP Top 10 — Injection (A03:2021); Brewer, CAP Twelve Years Later (IEEE Computer, 2012).